AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING

Travis Samson Fernandes
Department of Computer Engineering(of Aff.)
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce72.travis@pccegoa.edu.in

Mangesh Phadte
Department of Computer Engineering(of Aff.)
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce36.mangesh@pccegoa.edu.in

Chetan Naik
Department of Computer Engineering(of Aff.)
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce17.chetan@pccegoa.edu.in

Gaurang Madkaiker
Department of Computer Engineering(of Aff.)
Padre Conceicao College Of Engineering(Goa University)
Verna,Goa
21ce20.gaurang@pccegoa.edu.in

***Abstract*—The Automotive Diagnostic and Activity Monitoring (ADAM) System is an integrated solution designed to provide real-time insights into the health and behavior of four-wheeled vehicles. Acting as both a diagnostic tool and a forensic black box, ADAM harmonizes multiple sensing modules—including a Global Positioning System (GPS), a gyroscope and accelerometer (MPU6050), and a vibration sensor (SW-420)—to observe vehicular dynamics. At its core, the system interfaces with the vehicle's Electronic Control Unit (ECU) via the On-Board Diagnostics (OBD) port, enabling deep access to engine parameters and fault codes.Sensor data is continuously collected, securely stored on an SD card, and synchronized with a PostgreSQL database. This rich dataset fuels a suite of machine learning models tailored for four key objectives: (1) Engine Health Monitoring, (2) Fuel Economy Analysis, (3) Driver Behavior Profiling, and (4) Predictive Maintenance. Insights derived from these models are visually rendered through an intuitive mobile application that presents the information via dynamic graphs and charts.To further enhance user engagement, the system incorporates an interactive AI-powered chatbot capable of responding to user queries and guiding necessary actions. Together, these components form a cohesive ecosystem—bridging vehicle intelligence with user accessibility, and paving the way for smarter, safer, and more informed driving experiences.**

***Index Terms*—Raspberry pi 5, On Board Diagnostic(OBD), Global Positioning System(GPS), ECU(Electronic Control Unit)**

I. Introduction

Over the last couple of years, vehicle monitoring has been a necessary measure to make vehicles and drivers safe, cut down on emissions to tackle climate change , and provide optimal performance. The significance of protecting human life and the necessity to obtain genuine information at the time of accidents for forensic analysis reinforce the importance of efficient monitoring systems. To serve this purpose, we created a vehicle monitoring logger on the basis of On-Board Diagnostics (OBD-II) port. The OBD-II port is a 16-pin standardized connector found in every contemporary vehicle,developed in the United States Of America in 1996, by the Society of Automotive Engineers (SAE). It was imposed to enable regulation of vehicle emission so as to ensure Environmental Protection Agency(EPA) standards are met. It is a diagnostic port used to monitor real-time readings of vehicle sensors and the Diagnostic Trouble Codes (DTCs) from the Electronic Control Unit (ECU), making it a valuable resource for monitoring vehicle performance.Ecu is a the main controller of the vehicle where all controls and actions are done.

OBD II port supports communication protocols such as ISO15765-4 (CAN-BUS), ISO14230-4 (KWP2000), ISO 9141-2, SAE J1850 VPW, and SAE J1850 PWM. This study is centered on the ISO 15765-4 (CAN-BUS) protocol that has been required on vehicles manufactured from 1996. The message request is send in a hexadecimal to the protocols and it gets interpreted according to one of the five OBD II protocols and sends back a hexadecimal response.Obd II uses two types of codes to request ECU data that are Diagnostic Trouble Code (DTCs) and Parameter Identifiers (PIDs).DTC are used to diagnostic malfunctions in various subsystem of the vehicle and PIDs are used to measure real time parameters

II. Literature Review:

In Recent years, there has been advancements in design and development of On board Diagnostic and also there are machine learning algorithms for predicting engine performance , fuel consumption and anomaly detection. In this section, we review the literature on design and development of OBD boards and developed machine learning algorithms.Additionally, we identify gaps in recent literature

A. Design and Development of On-Boards Diagnostic circuits

The study [1] contributes to the design of On Board Diagnostic systems that provides real-time information about a vehicle, such as engine speed, coolant temperature, pressure, and Diagnostic Trouble Codes (DTCs).Data Collection of OBD Using a keaz128 microcontroller. This paper [2] designs

an Diagnostic system that has various data collection sources like gps,imu,camera and OBD port and stores locally for every 100 ms. Using BeagleBoard-XM with TI OMAP3530 processor logs the data. This paper [3] designs of Diagnostic system that collect obd data using ELM327 microcontroller along with gps .The data is transmitted to remote server and stored in sql database and transmit data using Carambola2 WiFi module.

B. Review of Machine Learning Algorithms

This study [4] is a detailed survey of machine learning methods used in engine performance, control, and fault diagnosis. It emphasizes the employment of Artificial Neural Networks (ANN), Support Vector Machines (SVM), Random Forests (RF), and neuro-fuzzy models such as ANFIS for improving fuel economy and emission control. The review also discusses the use of AI in engine control with a special focus on reinforcement learning (RL) and deep reinforcement learning (DRL) for enhanced adaptability, efficiency, and real-time decision-making in sophisticated engine operations. This paper [5] proposes an ML-based algorithm for automatic fault diagnosis in mobile networks using Self-Organizing Networks (SONs). It combines Softmax NN and SVM Models to carry out multiclass classification while diagnosing network faults. Along with KPIs and performance management counters (PMCs) for feature extraction. This unsupervised approach handles both single and multiple faults, offering a robust and adaptable solution for modern mobile networks. This paper [6] examines the use of artificial neural networks for the detection and classification of faults in internal combustion engines. It uses feedforward multilayer perceptron ANNs and training algorithms like backpropagation, Levenberg-Marquardt, quasi-Newton, extended Kalman filter, and a new Smooth Variable Structure Filter (SVSF). The study tested a method to detect engine faults without needing to know their exact location. Using vibration data from a V8 engine, converted into the crank angle domain, the system identified issues like missing bearings, chain tensioners and piston noise.Out of all the methods, SVSF, achieved 97% accuracy and avoided common training errors. It can detect and classify faults in under 30 minutes, offering a fast, low-cost solution for engine diagnostics. This paper [7] presents OBD-SecureAlert, a data-driven system that detects anomalies in vehicle behavior using CAN bus data.Since CAN buses lack built-in security measures, it is open to cyberattacks that will compromise the safety of automobiles. To overcome this situation the system uses Hidden Markov Model (HMM) to monitor and detect deviations of normal vehicle behaviour, emphasizing on detection rather than prevention. It collects data such as Speed and RPM, trains it using a sliding window and flags the data sets based on predefined threshold limits as anomalies or safety/cybersecurity threats. Attack scenarios are simulated, like injecting anomalous data into CAN bus in order to detect anomalies.

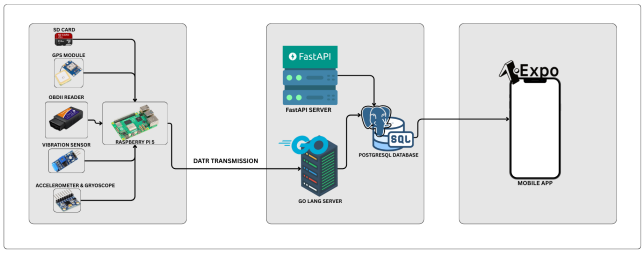

Fig. 1. System architecture of the proposed Automotive Diagnostic and Activity Monitoring (ADAM) system.

III. Methodology

A. System Architecture overview The proposed vehicle diagnostic and driver behaviour analysis system integrates various components of hardware and software to collect real time data collection for analysis and user analysis. The architecture is designed to collect vehicle information via obd II port, transmit it wirelessly to cloud server for processing and provides a visual representation of processed data through a mobile application.

The system consists of 4 primary components: Hardware Implementation Data processing using machine learning models cross platform mobile application A block diagram representing the overall system architecture and data flow from vehicle sensors to end user presentation is shown in figure 1.1

B. Hardware Implementation We have utilized an obd-II interface device compatible with standard protocols such as CAN(ISO 15765-4) and K-Line (ISO 9141-2). The device communicates with a raspberry pi 5 (a single-board computer),which is used for data collection along with other sensors like neo 6m gps module, sw 420 vibration sensor, mpu 6050 gyroscope/acceleration module. The data which is collected is stored in micro SD which can be used for forensic analysis.

C. Data Processing using machine learning model we have implemented four machine learning models that predict based on data are: driver behaviour analysis, fuel consumption, predictive maintenance, engine health performance

D. Cross platform mobile application The mobile application was developed using Expo (a cross platform framework), provides a dashboard for real time status, historical trends on fuel efficiency, driver behaviour and maintenance logs.

A. Model Training and Feature Importance Analysis

To classify engine conditions based on sensor readings, a Random Forest classifier was developed using an expert-labeled dataset named `Sensor-count_Breeza-Verna-Version4.xlsx`. The dataset underwent preprocessing, which involved trimming whitespace from column headers and removing incomplete records to ensure data quality.

The dataset was then divided into independent features and target labels, with the label column identified as `Lebelling`. Subsequently, an 80:20 train-validation split was applied to facilitate internal evaluation.

A Random Forest model comprising 100 decision trees was trained on the training subset. The model's predictive performance was assessed on the validation set using a classification report, which provided precision, recall, and F1-score metrics.

To interpret the model's decision-making process, feature importance scores were extracted. These scores indicate the relative influence of each sensor feature on the prediction outcome. A ranked list of features was generated, and the top ten most significant features were visualized using a horizontal bar plot.

Finally, the trained model was serialized and saved as `engine_model.joblib` for future use in prediction and deployment pipelines.

IV. DISCUSSION

Algorithm 1 Training and Evaluation of Engine Condition Classification Model

Input: Expert sensor dataset (`Sensor-count_Breeza-Verna-Version4.xlsx`)
Output: Trained classification model and feature importance ranking
Load the expert dataset from the Excel file
Clean the dataset by stripping whitespace from column names and removing rows with missing values
Separate the dataset into features (X) and target labels (y)
Split the dataset into training and validation sets using an 80:20 ratio
Initialize the Random Forest classifier with 100 estimators and a fixed random seed
Train the model using the training dataset
Predict the labels for the validation set
Evaluate the model's performance using a classification report
Compute the importance scores of each sensor feature using the trained model
Create a ranked list of features based on their importance scores
Visualize the top 10 most influential features using a bar plot
Save the trained model to disk as `engine_model.joblib`

Algorithm 2 Engine Condition Prediction and Summary Report

Input: Trained model file `engine_model.joblib`, test dataset `Breeza_bits.csv`
Output: Predicted labels, prediction summary, and overall engine condition status
Load the trained model from `engine_model.joblib`
Load the test dataset from `Breeza_bits.csv`
Clean the dataset by stripping column names and removing missing values
Retrieve and display the list of features expected by the model
Extract corresponding features from the test dataset based on the model's feature list
Predict the engine condition labels using the loaded model
Count the occurrences of each predicted label
Display the count of predictions per class label
Plot a bar chart to visualize the distribution of predicted labels
if both labels 'gg' and 'bb' are present in predictions **then**
    if count of 'gg' $>$ count of 'bb' **then**
        Report **Overall Condition: GOOD**
    else if count of 'bb' $>$ count of 'gg' **then**
        Report **Overall Condition: BAD**
    else
        Report **Balanced Condition: Equal 'gg' and 'bb' predictions**
    end if
else
    Display message indicating one or both labels were not detected in predictions
end if

V. RESULTS

A. Internal Evaluation on Expert Dataset

The trained Random Forest classifier was internally evaluated using an expert-labeled dataset. The dataset was partitioned into training and validation subsets using an 80:20 split. Table I presents the classification performance metrics obtained on the validation set.

TABLE I
CLASSIFICATION REPORT ON VALIDATION DATA

Class	Precision	Recall	F1-Score	Support
`bb`	0.85	0.85	0.85	48
`gg`	0.95	0.95	0.95	137
Accuracy	0.92 (185 samples)			
Macro Avg	0.90	0.90	0.90	185
Weighted Avg	0.92	0.92	0.92	185

The results indicate an overall accuracy of **92%**, with the `gg` class achieving slightly higher precision and recall compared to the `bb` class.

To identify the most influential sensor parameters contributing to the classification decisions, feature importance scores were extracted from the trained Random Forest model. Table II lists the top two most significant features.

TABLE II
TOP INFLUENTIAL FEATURES BASED ON IMPORTANCE SCORES

Feature	Importance Score
RUN_TIME	0.7149
Timestamp_OBD	0.2851

A visual representation of feature importance is depicted in Fig. 2. The plot highlights that RUN_TIME is the most dominant feature influencing the classification output.

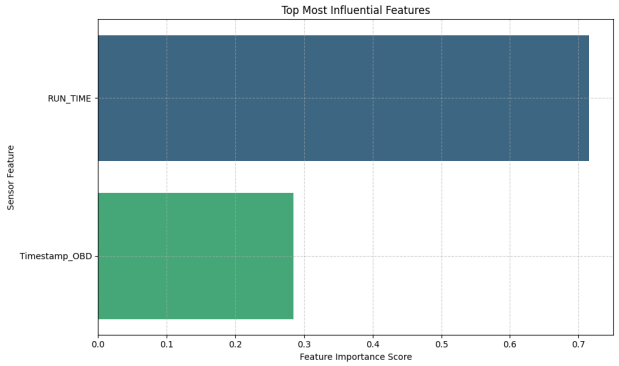

Fig. 2. Top Sensor Features Influencing Model Predictions

During visualization, a future deprecation warning from the Seaborn library was noted regarding the use of the palette parameter without an accompanying hue assignment. This issue will be addressed in subsequent revisions to maintain compatibility with future library updates.

Summary: The classifier demonstrated high predictive performance on expert data, with RUN_TIME emerging as the most critical sensor parameter influencing engine condition classification.

B. Prediction Results and Distribution Analysis

After deploying the trained engine condition classification model on real-world sensor data obtained from the Breeza_bits.csv dataset, the following results were observed.

The model expects two input features for prediction:

- Timestamp_OBD
- RUN_TIME

The test dataset was cleaned to remove missing values and the required features were extracted accordingly. Predictions were then performed using the loaded model.

The distribution of predicted class labels is presented in Table III. It indicates the frequency of each predicted condition label within the test dataset.

TABLE III
PREDICTION COUNT PER CLASS

Label	Count
gg (Good)	675
bb (Bad)	249

A visual representation of the prediction distribution is shown in Fig. 3. This bar chart highlights the number of occurrences for each predicted class label.

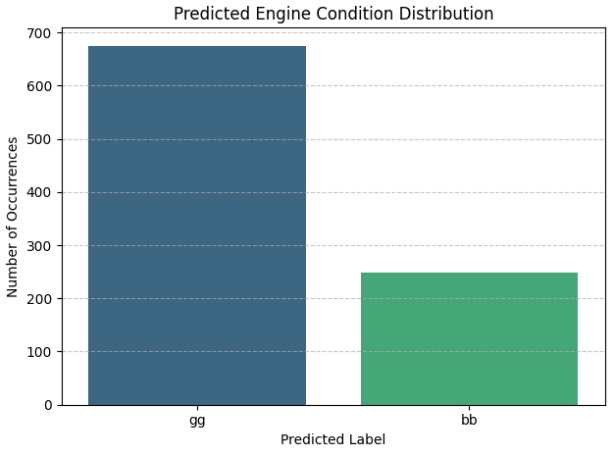

Fig. 3. Predicted Engine Condition Distribution

During visualization, a future deprecation warning was noted in the Seaborn library indicating that passing the palette parameter without a corresponding hue assignment will be deprecated in version 0.14.0. This will be addressed in future implementation updates to ensure compatibility.

Based on the prediction results:

- The majority of instances (675) were classified as gg (Good condition)
- 249 instances were classified as bb (Bad condition)

Hence, it can be concluded that the overall condition status of the test dataset is **Good**, as the number of healthy predictions outweighs the fault detections.

VI. CONCLUSION

The Automotive Diagnostic and Activity Monitoring (ADAM) System presents comprehensive, scalable, and intelligent diagnostics solutions. It integrates multiple hardware modules, machine learning model,s and a user-friendly mobile interface to offer real-time vehicle diagnostics, driver behavior analysis, and predictive maintenance. By combining OBD-II with additional sensor inputs and processing them using the Raspberry pi 5 it collects and stores data on the cloud and provides insights through a mobile app with multiple functionalities, including an AI chatbot. It not only aids in proactive maintenance, improved fuel efficiency, and personalized user experience but also contributes to safer and greener driving.

REFERENCES

[1] P. R. Sawant and Y. B. Mane, "Design and Development of On-Board Diagnostic (OBD) Device for Cars," in *Proc. 4th Int. Conf. on Computing Communication Control and Automation (ICCUBEA)*, Pune, India, 2018, pp. 1–4, doi: 10.1109/ICCUBEA.2018.8697833.

[2] D. L. Nguyen, M. E. Lee, and A. Lensky, "The design and implementation of new Vehicle Black Box using the OBD information," in *Proc. 7th Int. Conf. on Computing and Convergence Technology (ICCCT)*, 2012, pp. 1281–1284.

[3] R. Malekian, N. R. Moloisane, L. Nair, B. T. Maharaj, and U. A. K. Chude-Okonkwo, "Design and Implementation of a Wireless OBD II Fleet Management System," *IEEE Sensors Journal*, vol. 17, no. 4, pp. 1154–1164, Feb. 15, 2017, doi: 10.1109/JSEN.2016.2631542.

[4] L. F. I. Havugimana, B. Liu, F. Liu, J. Zhang, B. Li, and P. Wan, "Review of Artificial Intelligent Algorithms for Engine Performance, Control, and Diagnosis," *Energies*, vol. 16, no. 3, art. no. 1206, 2023, doi: 10.3390/en16031206.

[5] J. B. Porch, C. Heng Foh, H. Farooq, and A. Imran, "Machine Learning Approach for Automatic Fault Detection and Diagnosis in Cellular Networks," in *Proc. 2020 IEEE Int. Black Sea Conf. Commun. Netw. (BlackSeaCom)*, Odessa, Ukraine, 2020, pp. 1–5, doi: 10.1109/BlackSeaCom48709.2020.9234962.

[6] R. Ahmed, M. El Sayed, S. A. Gadsden, J. Tjong, and S. Habibi, "Automotive Internal-Combustion-Engine Fault Detection and Classification Using Artificial Neural Network Techniques," *IEEE Trans. Veh. Technol.*, vol. 64, no. 1, pp. 21–33, Jan. 2015, doi: 10.1109/TVT.2014.2317736.

[7] S. N. Narayanan, S. Mittal, and A. Joshi, "OBD SecureAlert: An Anomaly Detection System for Vehicles," in *Proc. 2016 IEEE Int. Conf. Smart Comput. (SMARTCOMP)*, St. Louis, MO, USA, 2016, pp. 1–6, doi: 10.1109/SMARTCOMP.2016.7501710.